

Chapter 2

How Computers Store, Move and Protect Data

Bits and bytes, file sizes, formats from CSV to Parquet, databases, servers and the cloud, APIs, and the habits that keep data safe.

Draft chapter

16 September 2026

Every size, output, and timing measured

Contents

Chapter 2. How Computers Store, Move and Protect Data

Why this matters

In plain English

2.1 Bits and bytes: how computers store everything

2.2 How big is big? From kilobytes to terabytes

2.3 Memory and storage: the desk and the shelves

2.4 Files, folders, paths, and extensions

2.5 Data file formats: the same data, packed six ways

2.6 Databases: data that many people can use at once

2.7 Servers, the internet, and the cloud

2.8 APIs: how systems talk to each other

2.9 Keeping data safe

Common mistakes and how to spot them

In the real world: the Friday file

Tools

The project: one dataset, five formats

You've got it when...

Recap

Practice exercises

Key terms

Where this leads

Answers to practice exercises

Chapter 2. How Computers Store, Move and Protect Data

Part 0 – First Principles: Data from Zero

Chapter at a glance

You will learn to: explain how a computer stores letters, numbers, and pictures as bits and bytes · read file sizes from bytes to terabytes, and work out how long a download takes · tell memory from storage · work with files, folders, paths, and extensions without surprises · choose between CSV, Excel, JSON, XML, PDF, and Parquet for a job, and avoid each format's traps · explain what a database, a server, the internet, and the cloud are · describe what an API does and read its replies · protect data with good passwords, encryption, access rules, and backups.

Before you start: Chapter 1 (what data is: rows, columns, types, and quality).

Time needed: 4–5 hours, including the exercises and the project.

Tools: any computer. For the project: a spreadsheet program (Excel or Google Sheets) and a plain-text editor (Notepad on Windows, TextEdit on a Mac, or any code editor).

Practice data: four Riverstone orders saved in five formats, and a demonstration API, both in the companion files (Appendix E). Every file size, output, and timing in this chapter comes from running those files.

Why this matters

In Chapter 1 you learned what data is. This chapter is about where it *lives*, how it *travels*, and how it's *kept safe*. That sounds like IT's job, and some of it is. But these everyday situations all land on a data person's desk:

- A colleague emails a CSV export, you open it in Excel, and every product code that started with zero has quietly lost it.
- A report that took two seconds to open last year now takes two minutes, and the file is 80 MB.
- The finance team's figures in the monthly deck don't match yours, because they're working from `Sales_Report_FINAL_v3.xlsx` and you're working from `Sales_Report_FINAL_v4.xlsx`.
- Your manager asks whether the new CRM "has an API" so the weekly report can update itself.
- A laptop with a customer list on it is left in a taxi.

None of these needs you to be an engineer. Each needs you to understand, in plain terms, how computers store, move, and protect data. People who do understand it waste less time, lose less data, and are trusted with more of it, which is exactly the path this book is about.

In plain English

Imagine Riverstone's old paper office before any computers arrived.

- Everything was written in an **alphabet** of letters and digits. A computer has an even smaller alphabet: just 0 and 1.
- Records were kept on **shelves** in a storage room, where they stayed even when the lights went off. That's a computer's **storage**: its disk.
- To work on a record, a clerk carried it to a **desk**. The desk was fast to work at but small, and cleared every evening. That's the computer's **memory**.
- Records sat in **folders with labels**, and the label said what kind of paper was inside. Those are **files**, **folders**, and **extensions**.
- The same information could be written as a **ledger** with neat columns, a **letter**, or a **form** with labeled boxes. Those are **file formats**.
- Documents traveled between branches by **courier**. That's the **network** and the **internet**.
- Other companies didn't walk into the storage room. They came to a **counter window**, filled in a request slip, and a clerk brought back exactly what they asked for. That's an **API**.
- The storage room had **locks**, only some staff had **keys**, and important ledgers were **copied** and kept in another building in case of fire. That's **security** and **backups**.

Every section below is one part of that office, in modern form.

2.1 Bits and bytes: how computers store everything

Everything is 0s and 1s

Deep down, a computer can only store and move **two states**: on or off, charged or not charged, magnetized one way or the other. We write those states as **1** and **0**. One of these on/off values is a **bit** (short for *binary digit*).

One bit can only tell two things apart: yes or no. But put bits together and the number of possibilities doubles with each bit you add:

Bits	Possible patterns	Enough for...
1	2	yes/no
2	4	the four seasons
8	256	every English letter, digit, and punctuation mark
16	65,536	a whole number up to 65,535
32	4,294,967,296	a whole number up to about 4.3 billion

A group of **8 bits** is a **byte**, and the byte is the basic unit for measuring data. When someone says a file is 364 bytes, they mean it takes $364 \times 8 = 2,912$ on/off switches to store.

How text is stored

To store text, computers agree on a table that gives every character a number, and store the number. The capital letter `A` is number 65, which in 8 bits is `01000001`. The digit `7` is number 55. A space is number 32. This agreement, first made for English as **ASCII** in the 1960s, fits every English letter, digit, and common symbol into one byte each.

English isn't the world's only language, of course. **Unicode** is the modern table: it gives a number to almost every character in every writing system, including ₹, Hindi, Tamil, Chinese, and emoji. The most common way to store Unicode as bytes is **UTF-8**, which keeps plain English letters at one byte each and uses two, three, or four bytes for everything else. Here's what some characters really take:

Text	Characters	Bytes in UTF-8
A	1	1
Pune	4	4
é	1	2
₹	1	3
न (Hindi <i>na</i>)	1	3
नमस्ते	6	18
😊	1	4
₹14,700	7	9

So "one character is one byte" is only true for plain English text. It's why a report with a lot of Hindi or Tamil text is bigger than the same report in English, and why a column limited to "100 characters" in one system can overflow a "100 bytes" limit in another.

When the table is wrong: garbled text

If a file is saved in one encoding and opened as if it were another, the bytes are read with the wrong table, and you get nonsense. Here's what happens when the UTF-8 bytes for `₹14,700` are opened as an older Windows encoding:

```
â, 14,700
```

And `Café` becomes `CafÃ©`. If you've ever seen `Ã©` or `â€™` in a report, a mismatched encoding is almost always the cause. The fix is to open or import the file with the correct encoding, which is usually UTF-8. Section 2.5 shows where this bites in practice.

How numbers are stored

Whole numbers are stored exactly, as binary. Numbers with decimals are trickier. Most programs store them in a format called **floating point**, which is fast but can only store most decimals *approximately*. Ask Python to add 0.1 and 0.2:

```
0.30000000000000004
```

The tiny error is invisible in most charts, but it's why $0.1 + 0.2 = 0.3$ can come out as *false* in a program, and why financial systems store money in exact decimal types instead (you'll meet `NUMERIC` in Chapter 12). Asked to add the same numbers as exact decimals, Python gives `0.3`.

How pictures and sound are stored

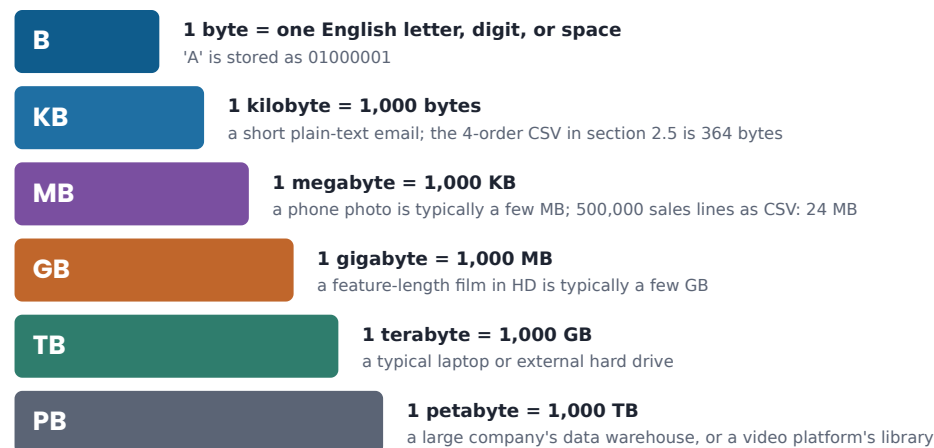
A digital photo is a grid of tiny dots called **pixels**. Each pixel stores three numbers, for how much red, green, and blue light it has, usually one byte each. A 12-megapixel phone photo has 12 million pixels, so before compression it takes $12,000,000 \times 3 = 36 \text{ MB}$. The file on your phone is usually only a few megabytes, because it has been **compressed** (section 2.5). Sound is stored in a similar way: thousands of measurements of the sound wave every second.

So text, numbers, photos, music, and video are all, in the end, long rows of 0s and 1s. What makes them different is the **agreement** about how to read those bits: the encoding or format.

Try it. Open any plain `.txt` file, count roughly how many characters it has, then check its size in bytes (right-click → *Properties* on Windows, or *Get Info* on a Mac). For English text, the two numbers should be close.

2.2 How big is big? From kilobytes to terabytes

Because data sizes range from a few bytes to many billions, we use prefixes, just as we say kilometers instead of thousands of meters.



Each step is 1,000 times the one before. (Some software counts in steps of 1,024; see section 2.2.)

Figure 2.1 – Each step is a thousand times bigger. The examples are typical sizes, not fixed rules.

A few sizes worth remembering: a page of plain English text is about 2–3 KB; a phone photo is typically a few MB; an hour of HD video is typically a few GB; and Riverstone's test file of 500,000 sales lines, which you'll meet in section 2.5, is 24 MB as a CSV.

Why your 1 TB drive shows 931 GB

There are two ways to count, and both are in use:

- **Decimal (the SI standard):** 1 KB = 1,000 bytes, 1 MB = 1,000,000 bytes, 1 GB = 1,000,000,000 bytes. Drive makers and most network speeds use this.
- **Binary:** 1 "KB" = 1,024 bytes (2^{10}), 1 "MB" = $1,024 \times 1,024$ bytes, and so on. Windows, and some older software, count this way.

The binary units have their own proper names, **KiB, MiB, GiB** (kibibyte, mebibyte, gibibyte), though few people use them in conversation. The difference grows with size. A drive sold as 1 TB holds 1,000,000,000,000 bytes. Divide by $1,024^3$ (the binary gigabyte) and you get **931.3**, which is the "GB" Windows displays. No space is missing; it's the same number of bytes counted in a different unit.

Bits for speed, bytes for size

Internet speeds are quoted in **megabits per second (Mbps)**, with a small *b*. File sizes are in **megabytes (MB)**, with a capital B. Since a byte is 8 bits, divide by 8:

- A 100 Mbps connection moves at most $100 \div 8 = 12.5$ **MB per second**.
- Downloading a 5 GB file on it takes at least $5,000 \text{ MB} \div 12.5 \text{ MB/s} = 400$ **seconds**, about **6.7 minutes**. In practice it takes longer, because the connection is shared and rarely runs at full speed.

Watch out: b and B. "My file is 80 Mb" and "my file is 80 MB" differ by a factor of eight. In data work, sizes are almost always bytes (MB, GB); speeds are almost always bits (Mbps, Gbps). When a number looks eight times too big or too small, check the letter.

Size limits you'll actually hit

- **Excel** holds at most **1,048,576 rows** and **16,384 columns** per sheet. Long before that, a large workbook becomes slow to open and save. A dataset with millions of rows belongs in a database (Chapter 12) or a format such as Parquet (section 2.5).
- **Email attachments** are usually limited to a few tens of megabytes, which is one reason teams share large files by link instead.
- **Cloud storage plans, phone storage, and laptop drives** are all limited, and duplicate copies of large files (`report_v1` , `report_v2` , `report_final` ...) fill them quickly.

2.3 Memory and storage: the desk and the shelves

Every computer has two very different places to keep data.

	Memory (RAM)	Storage (SSD or hard disk)
Office analogy	the desk you're working at	the shelves in the storage room
What it holds	whatever you have open right now	every file, program, and photo you've saved

	Memory (RAM)	Storage (SSD or hard disk)
Speed	very fast	slower (SSDs are much faster than older spinning hard disks)
Size on a typical laptop	about 8–32 GB	about 256 GB–2 TB
When the power goes off	everything in it is lost	everything stays

Three everyday consequences:

1. **Unsaved work lives only in memory.** If Excel crashes before you save, the changes since your last save were never on the shelves. Save often, or turn on your program's autosave.
2. **Opening a file copies it into memory.** A 200 MB workbook needs well over 200 MB of RAM to work with, often several times more once formulas and formatting are loaded. When a big spreadsheet slows your whole computer, memory is usually what's run out.
3. **Where "saved" means.** Your file might be saved on your own computer's drive, on a **shared network drive** at the office, or in **cloud storage** such as Google Drive or OneDrive, which keeps a copy on the provider's computers and **syncs** it to your devices. Knowing which one matters when you ask, "Can my colleague see this?" or "Is this backed up?"

2.4 Files, folders, paths, and extensions

A **file** is a named chunk of data on storage. A **folder** (or *directory*) is a container for files and other folders. Every file has a **path**, its full address, which lists every folder you pass through to reach it:

```
Windows: C:\Users\Kavya\Documents\Finance\2026-09_spending_log.xlsx
macOS:   /Users/kavya/Documents/Finance/2026-09_spending_log.xlsx
```

Windows separates folders with a backslash `\` and starts with a drive letter such as `C:`. macOS and Linux use a forward slash `/`. You'll type paths when you load files into SQL, Python, or Power BI, and most "file not found" errors are a path with one folder or one letter wrong.

Extensions: what kind of file is this?

The letters after the last dot in a file name are its **extension**. The extension tells the computer which program should open the file:

Extension	What it usually is
<code>.txt</code>	plain text
<code>.csv</code>	comma-separated values: a table as plain text
<code>.xlsx</code>	an Excel workbook (<code>.xls</code> is the older format; <code>.xlsm</code> can contain macros)
<code>.json</code> , <code>.xml</code>	structured text, often exchanged between systems
<code>.pdf</code>	a document laid out for reading and printing

Extension	What it usually is
.jpg , .png	images
.parquet	a compressed, column-based data file (section 2.5)
.sql	a text file of SQL statements
.py	a Python program
.zip	a compressed bundle of other files
.exe	a Windows program that runs when you open it

Renaming `sales.csv` to `sales.xlsx` doesn't convert it into an Excel file. It only changes the label, and Excel will warn that the file doesn't match its name. To convert, open the file and use *Save As*.

Watch out: hidden extensions. Windows hides known extensions by default, so a file named `invoice.pdf.exe` appears as `invoice.pdf`, with a harmless-looking name and a program hidden inside. It's a classic trick in phishing emails. Turn on *File name extensions* in File Explorer's *View* menu, and never open an unexpected attachment that turns out to end in `.exe`, `.js`, `.bat`, `.scr`, or `.xls` from someone you don't know.

Naming files so people can find them

Compare these two folders:

Report final.xlsx	2026-01-31_monthly_sales_report.xlsx
Report final (2).xlsx	2026-02-28_monthly_sales_report.xlsx
Report FINAL v3 use this one.xlsx	2026-03-31_monthly_sales_report.xlsx

The right-hand names follow four habits: **a date first, written year-month-day**, so files sort in time order automatically (Chapter 1); **the same pattern every time**; **lower-case words joined with underscores or hyphens**, which avoids problems in code and web links; and **no words like "final"**, which are always eventually wrong. If you need versions, use the version history in Google Drive, OneDrive, or SharePoint, or, for queries and code, Git (Chapter 26).

2.5 Data file formats: the same data, packed six ways

Riverstone's sales coordinator exports four February orders. Here is exactly the same data saved in five data formats. Look at what each one looks like inside; the differences explain when to use each.

CSV: a table as plain text

```
order_id,customer_name,order_date,status,sales_rep,net_revenue,delivery_note
5006,Metro Mart,2026-02-06,Delivered,Rahul Mehta,14640.00,
5007,Coastal Foods,2026-02-11,Delivered,Rahul Mehta,32625.00,Call before delivery
5008,Sunrise Caterers,2026-02-19,Delivered,,23325.00,
5009,Northgate Distributors,2026-02-25,Shipped,Farah Khan,76560.00,"Gate 2, Okhla Phase II"
```

CSV (comma-separated values) is the simplest data format there is: one line per row, commas between values, and usually a first line of column names. Any spreadsheet, database, and programming language can read it, which is why "export to CSV" is the universal language between systems. This file is **364 bytes**.

Look closely and you'll find three of CSV's limits:

1. **Commas inside values need quotes.** Northgate's delivery note contains a comma, so it's wrapped in double quotes. Without them, a program would see an extra column.
2. **A blank is just nothing.** Order 5008's missing sales rep is two commas in a row. CSV can't tell "unknown", "not applicable", and an empty text value apart (Chapter 1).
3. **There are no types.** Everything in a CSV is text. When a program reads this file back, `2026-02-06` arrives as text, not a date, unless the program guesses or you tell it. Reading it back with Python's pandas library, `order_date` comes back as text while `net_revenue` is guessed to be a number.

Watch out: opening a CSV in Excel changes it. When Excel opens a CSV directly, it guesses a type for every value, and some guesses silently change your data: product codes like `00451` lose their leading zeros and become `451`; long ID numbers turn into `4.52E+13`; codes like `3-4` or `1/2` can become dates. If you then save, the damage is written back into the file. Instead, use *Data → From Text/CSV* (Excel) or *File → Import* (Google Sheets), and set code and ID columns to **Text** before loading. If ₹ or accented names appear garbled, choose **UTF-8** as the file's encoding in that same import window (section 2.1).

Excel (.xlsx): a workbook for people

An `.xlsx` file looks like a grid in Excel, but it isn't plain text. It's actually a **zip file** holding a small collection of XML files. Unzip the four-order workbook and you find ten files inside, including:

```
[Content_Types].xml
xl/workbook.xml
xl/worksheets/sheet1.xml
xl/sharedStrings.xml
xl/styles.xml
```

That structure lets a workbook hold far more than a CSV: several sheets, real dates and numbers, formulas, formatting, charts, and pivot tables. It also means the file isn't meant to be read as text, and needs software that understands the format. This tiny workbook is **5,674 bytes**, more than fifteen times the CSV, because the structure itself takes space. Excel is the right choice when **people** will open, read, and work with the file.

JSON: data with labels, for systems

```
[
  {
    "order_id": 5006,
    "customer_name": "Metro Mart",
    "order_date": "2026-02-06",
    "status": "Delivered",
    "sales_rep": "Rahul Mehta",
    "net_revenue": 14640.0,
    "delivery_note": null
  },
  {
    "order_id": 5008,
    "customer_name": "Sunrise Caterers",
    "order_date": "2026-02-19",
    "status": "Delivered",
    "sales_rep": null,
    "net_revenue": 23325.0,
    "delivery_note": null
  }
]
```

(Two of the four orders shown; the file has all four.)

JSON (JavaScript Object Notation) is the semi-structured format from Chapter 1. Each record is a set of **"name": value** pairs inside curly braces `{ }`, and a list of records goes inside square brackets `[ ]`. Notice what JSON can express that CSV can't: text is in quotes and numbers aren't, so some types survive; and a missing value is written as `null`, clearly different from an empty text value `" "`. A record can also contain a list, such as an order with its order lines inside it. JSON is how most **apps and APIs** exchange data (section 2.8). Because every record repeats every column name, it's larger: **894 bytes** here.

XML: labels as tags

```
<order id="5009">
  <customer_name>Northgate Distributors</customer_name>
  <order_date>2026-02-25</order_date>
  <status>Shipped</status>
  <sales_rep>Farah Khan</sales_rep>
  <net_revenue>76560.00</net_revenue>
  <delivery_note>Gate 2, Okhla Phase II</delivery_note>
</order>
```

XML (eXtensible Markup Language) does the same job as JSON with **tags**: every value sits between an opening `<name>` and a closing `</name>`, and an empty value can be written `<sales_rep/>`. It's older and wordier (the four-order file is **1,118 bytes**), and you'll meet it in banking and government systems, e-invoicing, older business software, and inside Office files like the workbook above.

PDF: made for reading, not for data

A **PDF** fixes exactly how a document looks on screen and on paper, which makes it ideal for invoices, contracts, and reports people read. It's poor for data: a table in a PDF is stored as text placed at positions on a page, not as rows and columns, so copying it into a spreadsheet often scrambles the

columns. When someone offers you data "as a PDF", ask whether the CSV or Excel export behind it exists. (When it doesn't, Chapter 58 shows how AI tools can extract tables from documents, with checks.)

Parquet: built for large-scale analysis

Parquet is a format built for analyzing large datasets, and it's the standard format of cloud data platforms (Part V). You can't read it as text: the file begins with the four letters `PAR1`, and the rest is compressed binary data. What makes it special is how it's laid out.

CSV and Excel: stored row by row

2025-08-30	2	105	77	row 1
2025-02-24	21	107	62	row 2
2025-11-03	7	101	15	row 3
2025-05-19	2	103	140	row 4

To add up qty, the computer still reads every value in every row, then throws most away.

Parquet: stored column by column

date	2025-08-30	2025-02-24	2025-11-03	2025-05-19
customer	2	21	7	2
product	105	107	101	103
qty	77	62	15	140

To add up qty, it reads only the qty column. Similar values sit together, so they compress well.

Figure 2.2 — CSV stores data row by row; Parquet stores it column by column.

A CSV stores **row by row**: order 1's date, customer, product, quantity; then order 2's. Parquet stores **column by column**: all the dates together, then all the customers, then all the quantities. Two big benefits follow. To total the quantity column, a program reads only that column and skips the rest. And values in one column are similar to each other (a column of dates, a column of product IDs), so they **compress** very well. Parquet also stores each column's **type**, so dates come back as dates.

For four orders, Parquet's advantages are invisible: the file is **4,872 bytes**, bigger than the CSV, because of the information it stores about its own structure. Its advantages appear at scale.

The same test at scale

To see the differences properly, the same 500,000 sales lines, with 8 columns, were saved in each format and read back with Python on a small two-processor cloud computer:

Format	File size	Time to read everything	Time to read one column
CSV	24.1 MB	0.23 s	0.10 s
CSV, compressed with gzip	4.7 MB	—	—
JSON	82.6 MB	1.43 s	—
Excel (.xlsx)	18.3 MB	31.4 s	—
Parquet	4.3 MB	0.02 s	0.003 s

Exact times depend on the computer, but the pattern holds everywhere. **Parquet was the smallest file, and about ten times faster than CSV to read in full.** Excel took more than a hundred times longer than CSV to read, because every cell has to be unpacked from zipped XML. JSON was the

largest, because it repeats all eight column names on every one of the 500,000 rows.

Compression

Compression makes files smaller by writing repeated patterns more efficiently. Zipping the CSV cut it from 24.1 MB to 4.7 MB, because a column like `status` repeats the same few words half a million times. There are two kinds:

- **Lossless** compression (ZIP, gzip, and the compression inside Parquet and `.xlsx`) gives back **exactly** the original bytes when you unpack it. Data files must only ever use lossless compression.
- **Lossy** compression (JPEG photos, MP3 music, most video) throws away detail people are unlikely to notice. It's fine for a photo, and unacceptable for a sales ledger.

Choosing a format

Format	People can read it as text?	Keeps types?	Size	Opens in Excel?	Best for
CSV	yes	no	small	yes (import carefully)	moving tables between systems; simple exports
Excel	no (needs Excel or similar)	yes	medium	yes	files people will read, filter, and work in
JSON	yes	partly	large	with Power Query	apps and APIs; nested records
XML	yes	partly	large	with Power Query	older systems, e-invoicing, Office internals
PDF	no	no	varies	no	documents for reading and printing
Parquet	no	yes	smallest	no (Power BI and Python can read it)	large datasets for analysis

Real-life example: why the data team asks for "the raw export". A manager sends a PDF of last quarter's sales, laid out beautifully. To analyze it, the analyst has to copy 40 pages of tables by hand or with a converter, and check every row. The same data exported from the billing system as CSV takes ten seconds to load. The rule most data teams follow: **PDF and formatted Excel for people who read; CSV, JSON, or Parquet for machines that process.**

2.6 Databases: data that many people can use at once

Files are fine for one person and one moment in time. Business data needs more. Picture Riverstone's orders kept as a shared Excel file: three salespeople edit it at once, two copies drift apart, someone sorts one column without the others and scrambles every order, and the file grows until it no longer opens.

A **database** is software built to avoid those problems. It keeps data in **tables** (rows and columns, like a strict spreadsheet), and it:

- lets **thousands of people and programs** read and write at the same time without overwriting each other;
- enforces **rules**: no order for a customer who doesn't exist, no negative quantities, no duplicate invoice numbers;
- answers **questions** in a language called **SQL**, even across millions of rows, in seconds;
- controls **who may see or change** each table;
- keeps a record of changes so it can recover after a crash.

Most business databases run as a **database server**: a program on a computer, usually in a data center or the cloud, that other programs connect to over the network. (A few, like SQLite, which runs inside many phone apps, store the whole database in a single file.) Underneath, a database still saves its data in files on storage; you just never touch those files directly.

This one page is only a preview. **Chapter 12** teaches databases and SQL properly, from creating your first table onward.

2.7 Servers, the internet, and the cloud

Clients and servers

When you open Gmail, two computers are involved. Your laptop or phone is the **client**: it asks for something. A computer in one of Google's data centers is the **server**: it listens for requests and answers them. A server isn't a special kind of machine; it's a computer whose job is to serve requests, usually running all day in a **data center**, a building full of such computers with reliable power, cooling, and network connections.

How a request finds its way

The **internet** is a worldwide network of networks that carries data between clients and servers. Three ideas explain most of it:

- **IP address**. Every device on a network has an address, such as `203.0.113.14`, so data knows where to go, like a postal address.
- **DNS** (Domain Name System). People remember names like `riverstone.example`, not numbers. DNS works like a phone book, turning a name into an IP address before your request is sent.
- **HTTP and HTTPS**. These are the rules for how a browser or program asks a web server for something and gets a reply. **HTTPS** is the secure version: everything sent is **encrypted**, so people along the route can't read or change it. The padlock icon in a browser means the connection uses HTTPS. It does *not* mean the website itself is trustworthy.

Data on the internet travels in small **packets**, each finding its own route, then reassembled at the other end. You never see that, but it's why a large file arrives in pieces and why a slow connection affects everything at once.

The cloud

"The cloud" sounds mysterious. It's simpler than it sounds: **the cloud is computers in someone else's data center that you rent over the internet**, instead of buying and running your own. The largest providers include Amazon Web Services (AWS), Microsoft Azure, and Google Cloud.

Businesses rent at three levels, depending on how much they want to manage themselves:

Level	You rent...	You still manage...	Examples
IaaS (infrastructure as a service)	virtual computers, storage, and networks	the operating system, software, and data	a virtual server to run your own database
PaaS (platform as a service)	a ready-to-use platform, such as a managed database	your data and how you use it	a cloud PostgreSQL service that handles backups and updates for you
SaaS (software as a service)	a finished application, used through a browser	your data and your settings	Gmail, Google Drive, Microsoft 365, CRM and accounting software

Why companies move to the cloud: they **pay for what they use** instead of buying servers upfront, they can **grow or shrink** in minutes, and the provider handles power, hardware failures, and much of the security. The trade-offs: bills that grow quietly if nobody watches them (Chapter 65), dependence on one provider, and questions about **where the data is physically stored**. Many companies, and some laws, require certain data to stay in a particular country, so cloud services let customers choose a **region** for their data.

2.8 APIs: how systems talk to each other

Riverstone's sales system, its accounting software, its website, and its payment gateway are separate programs, often from separate companies. For a website order to appear in the sales system, or for a weekly report to pull the latest invoices, the programs need a way to ask each other for data. That way is an **API** (Application Programming Interface).

Like a restaurant: you (the program) never walk into the kitchen (the database). You give the waiter (the API) a clear order, and it brings back a dish.

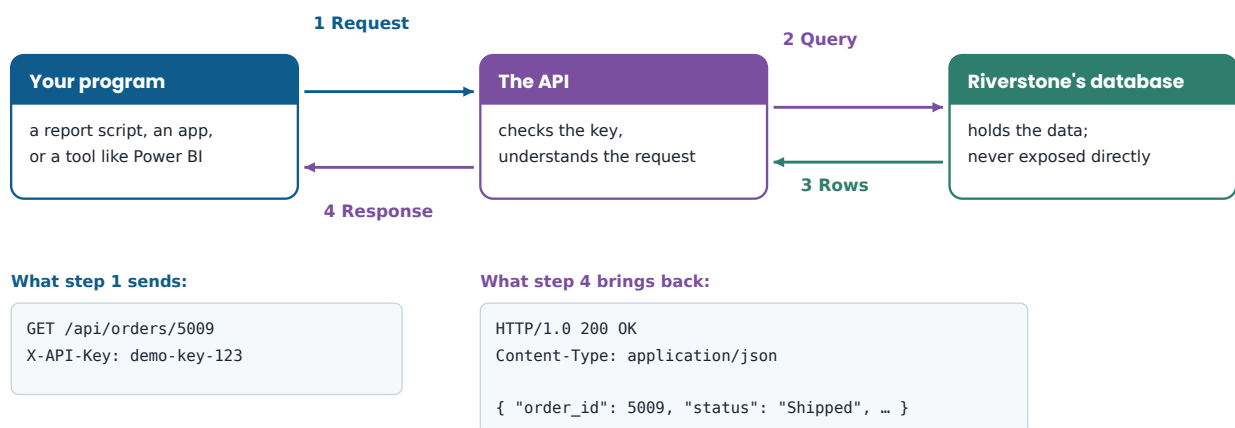


Figure 2.3 — An API is the waiter between your program and someone else's kitchen.

The restaurant analogy is the classic one. You don't walk into the kitchen and take food off the stove; you'd get in the way, and you might take someone else's dish. You give a **waiter** a clear order from the **menu**, and the waiter brings back what you asked for. An API is the waiter: it offers a menu of requests it will accept, checks you're allowed to ask, fetches what's needed from the system behind it, and brings back a reply in an agreed format, usually JSON. The system's database (the kitchen) is never opened up to outsiders.

A real request and response

The companion files include a tiny demonstration API for Riverstone that runs on your own computer (Appendix E). A **request** asks for one order by its address, and includes an **API key**, a secret code that proves the caller is allowed in. Here's the full reply to a request for order 5009, sent with the `curl` command-line tool:

```
HTTP/1.0 200 OK
Server: RiverstoneAPI/1.0
Date: Wed, 16 Sep 2026 10:30:00 GMT
Content-Type: application/json
Content-Length: 147

{
  "order_id": 5009,
  "customer_name": "Northgate Distributors",
  "order_date": "2026-02-25",
  "status": "Shipped",
  "net_revenue": 76560.0
}
```

Every **response** has two parts. The top is the **header**: a **status code** (200 OK) that says how the request went, plus details such as the type of content coming back. After a blank line comes the **body**: the data itself, here in JSON.

Ask for an order that doesn't exist, and the status code changes:

```
HTTP/1.0 404 Not Found
Server: RiverstoneAPI/1.0
Date: Wed, 16 Sep 2026 10:30:00 GMT
Content-Type: application/json
Content-Length: 38

{
  "error": "order 5099 not found"
}
```

Leave out the API key, and the API refuses before it even looks for the order:

```
HTTP/1.0 401 Unauthorized
Server: RiverstoneAPI/1.0
Date: Wed, 16 Sep 2026 10:30:00 GMT
Content-Type: application/json
Content-Length: 44

{
  "error": "missing or invalid API key"
}
```

Status codes worth knowing

The first digit tells you who's responsible: **2** means it worked, **4** means the request has a problem (your side), **5** means the server has a problem (their side).

Code	Meaning	What to do
200 OK	it worked	read the body
201 Created	a new record was created	note the new record's ID
400 Bad Request	the request is malformed	check what you sent
401 Unauthorized	no valid key or login	check the API key
403 Forbidden	you're recognized, but not allowed to do this	ask for the right permission
404 Not Found	no such record or address	check the ID and the address
429 Too Many Requests	you've hit the API's rate limit	slow down and retry later
500 Internal Server Error	the server failed	retry later; tell the API's owner if it persists

Why APIs matter to data people

Most modern business software, including CRMs, accounting tools, payment gateways, e-commerce platforms, and ad platforms, offers an API. That's what makes automation possible: a script can pull yesterday's invoices every morning without anyone logging in and clicking *Export*, and a dashboard can refresh itself. You'll call real APIs from Python in Chapter 18, automate reports with them in Chapter 20, and design how whole systems exchange data in Chapter 51.

Two more terms you'll hear:

- A **webhook** is an API in reverse. Instead of your program asking "anything new?" every five minutes, the other system calls *your* address the moment something happens, such as "payment received".
- **API keys are passwords**. Anyone who has the key can do whatever the key allows. Never paste one into a spreadsheet, a shared document, a chat, or code you publish online. Chapter 20 shows how to store them safely.

2.9 Keeping data safe

The three goals: confidentiality, integrity, availability

Security professionals describe what they protect with three words, often called the **CIA triad**:

Goal	Means	Example of failure at Riverstone
Confidentiality	only the right people can see the data	the customer price list is emailed to a competitor by mistake

Goal	Means	Example of failure at Riverstone
Integrity	the data is accurate and hasn't been changed without authorization	someone edits invoice amounts in a shared spreadsheet, and nobody can tell
Availability	the data is there when people need it	ransomware locks the sales server during the busiest week of the year

Every habit below protects one or more of the three.

Accounts and passwords

Most data breaches start with a person, not a clever technical attack: a reused password, a shared login, or a click on a fake email. The habits that prevent most of them:

- **Long and unique.** A long passphrase is harder to guess than a short password full of symbols, and **every account needs a different one**. When one website is breached, attackers try the same email and password everywhere else.
- **Use a password manager.** Nobody can remember fifty unique passphrases. A password manager remembers them, fills them in, and warns you about reused ones.
- **Turn on multi-factor authentication (MFA).** A second step, such as a code from an authenticator app or a prompt on your phone, stops most attacks even when a password leaks.
- **Never share logins.** A CRM password shared in a team chat means you can't tell who changed what, and you can't remove one person's access without locking everyone out. Each person gets their own account.
- **Be suspicious of urgency.** Phishing emails pressure you to act now: "Your account will be closed", "Pay this invoice today". Check the sender's real address, hover over links before clicking, and confirm unusual payment requests by phone.

Access: the principle of least privilege

Give each person and program only the access they need, and nothing more. An analyst who builds sales reports needs to *read* the sales tables, not change them, and doesn't need salary data at all. That's why, as Chapter 12 mentions, many companies give analysts **read-only** accounts. Least privilege limits the damage from a mistake or a stolen password.

Encryption

Encryption scrambles data with a key so that only someone with the right key can read it. You rely on it in two places:

- **In transit**, while data moves: HTTPS websites, secure email, and encrypted connections to databases.
- **At rest**, while data sits on storage: an encrypted laptop drive (BitLocker on Windows, FileVault on a Mac) means a stolen laptop's files are unreadable without your login. For any laptop that carries work data, this should be switched on.

Integrity checks: fingerprints for files

How do you know a file hasn't been changed, even by one character? Computers calculate a **hash**: a fixed-length "fingerprint" of the data. The same data always gives the same hash, and the smallest change gives a completely different one. Here are the SHA-256 hashes of two payment instructions that differ only in the order of two digits:

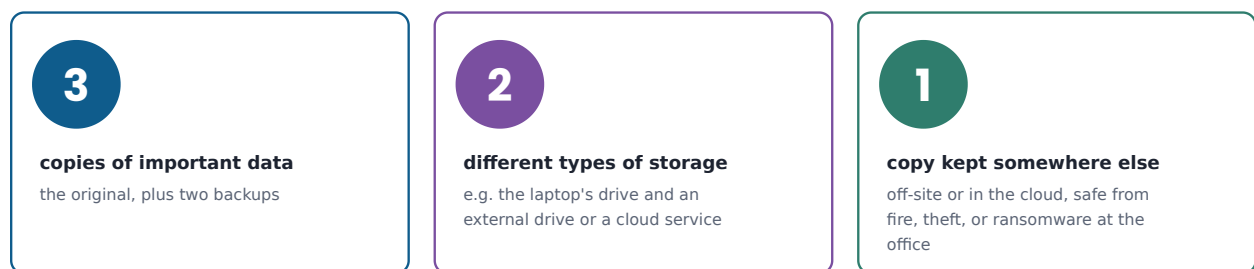
```
Text:    Pay Rs 14,700 to Riverstone Supplies
SHA-256: f4251ff3fb7191f7e79677f3b1871db06c0935c3cc08164496995f1f909f6f71

Text:    Pay Rs 17,400 to Riverstone Supplies
SHA-256: 9d2f842c50110e140abd578e7e8460d1f6f2bbca7eba341ce1e5bee43566d0fa
```

Nothing about the second fingerprint resembles the first. Software uses hashes to check that downloads arrived undamaged, that backups match the original, and that passwords are stored safely (systems store a hash of your password, not the password itself). Data pipelines use them to detect whether a file has changed since yesterday (Chapter 45).

Backups and versions

Hardware fails, laptops are stolen, files are deleted by mistake, and **ransomware** (malicious software that locks files and demands payment) can encrypt everything a computer can reach. The only complete protection is a copy you can restore. The widely used rule of thumb is **3-2-1**:



A backup you have never restored is a hope, not a backup: test a restore regularly.

Figure 2.4 – Three copies, two kinds of storage, one kept elsewhere.

Two points people often miss:

- **Syncing is not backing up.** Cloud storage such as Google Drive or OneDrive **copies every change** to every device, including deleting a file or saving over it with a mistake. Version history and a recycle bin help, but only for a limited time. A true backup is a separate copy that your everyday mistakes don't touch.
- **Test your restore.** Many organizations discover their backups were incomplete only on the day they need them.

Version history is backup's everyday cousin. Google Drive, OneDrive, and SharePoint keep earlier versions of a file, so you can see who changed what and roll back a bad edit. Use it instead of saving `report_v7_FINAL.xlsx`. For SQL queries and code, Git does the same job more precisely (Chapter 26).

Personal data

Some data is about **people**: names, phone numbers, addresses, ID numbers, health information, salaries. Handle it with extra care. India's Digital Personal Data Protection Act, 2023, the European Union's GDPR, and similar laws elsewhere set rules for collecting, storing, and sharing it. The everyday habits: collect and keep only what you need; don't copy it to personal devices, personal email, or personal cloud accounts; share it only with people entitled to see it; and use anonymized or sample data for practice and demos. Chapter 64 covers privacy and governance in depth.

If you make a mistake, say so immediately. Emailed a customer list to the wrong person? Clicked a suspicious link? Lost a laptop? Tell your manager and IT **straight away**. Minutes matter: a password can be reset, a sent email sometimes recalled, a laptop wiped remotely. Organizations forgive a reported mistake far more easily than a hidden one.

Common mistakes and how to spot them

Mistake	Symptom	Fix
Opening a CSV directly in Excel and saving it	leading zeros gone, IDs as 4.52E+13 , codes turned into dates	Import with <i>Data</i> → <i>From Text/CSV</i> ; set ID columns to Text
Wrong text encoding	â,¹, Ã©, â€™ in names and amounts	Import as UTF-8
Confusing Mbps with MB/s	downloads "eight times slower than expected"	Divide Mbps by 8 for MB/s
Renaming a file's extension to "convert" it	the file won't open, or opens with a warning	Open the file and use <i>Save As</i>
Using "final", "new", or "(2)" in file names	nobody knows which file is current	Date-first names plus version history
Sending data as PDF	the receiver re-types or mis-copies tables	Share the CSV or Excel export
Keeping millions of rows in Excel	files take minutes to open; rows cut off at 1,048,576	Use a database or Parquet
Pasting API keys or passwords into files and chats	anyone with the file can use the account	Store secrets in a password manager or secure settings
Sharing one login among a team	no record of who changed what	One account per person, with only the access they need
Treating cloud sync as a backup	a deletion or a bad save spreads to every copy	Keep separate, tested backups (3-2-1)
Unencrypted work laptop	a lost laptop becomes a data breach	Turn on drive encryption
Hiding a security mistake	a small incident becomes a large one	Report it immediately

In the real world: the Friday file

Before Meera (Chapter 1) joined, Riverstone's weekly sales report worked like this. Every Friday afternoon, Imran, who ran sales operations, exported orders from the billing system as CSV, opened the file in Excel, added a few formulas and a pivot table, and saved it as `Weekly Sales FINAL.xlsx` on his laptop. Then he emailed it to eleven people. Over one bad month, four things went wrong:

1. **Product codes broke.** Riverstone's new product range used codes such as `00731`. Excel turned them into `731`, and the lookup to the price list failed for every new product. Two weeks of reports understated new-product sales.
2. **Nobody knew which file was current.** Imran sent a corrected report on a Monday, as `Weekly Sales FINAL (2).xlsx`. Half the recipients kept using the Friday version, and the regional managers argued about numbers that came from two different files.
3. **The file grew until it choked.** Each week's data was pasted under the last. At 40 MB the workbook took four minutes to open, and some recipients' email rejected it.
4. **The laptop was stolen** from a car. It wasn't encrypted, and the workbook held two years of customer names, phone numbers, and prices.

Riverstone's fix used nothing more advanced than this chapter:

- The export was **imported** with codes set to text, and the billing system's **API** later replaced the manual export entirely.
- The report moved to **one shared location** with version history, and the email became a **link** to it, with the date in the file name. Everyone saw the same, current file.
- History moved out of the workbook into a **database**, and the report kept only the latest weeks, so it opened in seconds.
- Every company laptop was **encrypted**, customer phone numbers were removed from reports that didn't need them, and the IT team set up **tested 3-2-1 backups**.

None of those changes required a data engineer. They required someone who understood files, formats, APIs, and basic security well enough to notice what was wrong. Chapter 20 takes this exact report and automates it end to end.

Tools

- **A plain-text editor.** Notepad (Windows), TextEdit in plain-text mode (Mac), or a free code editor such as Visual Studio Code. Opening a CSV or JSON file in a text editor shows you what's really inside, without a spreadsheet's guesses.
- **Excel or Google Sheets**, for the project. Learn the *import* routes (*Data* → *From Text/CSV* in Excel; *File* → *Import* in Google Sheets), not just double-clicking.
- **A password manager and an authenticator app.** Set them up for your own accounts this week.
- **The companion files** (Appendix E): `orders_feb_2026` in `.csv`, `.xlsx`, `.json`, `.xml`, and `.parquet`; and `api_demo.py`, the demonstration API used in section 2.8, which you'll run yourself in Chapter

The project: one dataset, five formats

Goal: see with your own eyes what each format stores, what it loses, and how each program treats it.

Option A: use the companion files `orders_feb_2026.csv`, `.xlsx`, `.json`, and `.xml`. **Option B:** use your spending log from Chapter 1's project, and create the formats yourself: save it from your spreadsheet as `.xlsx` and as CSV (UTF-8), then type a JSON version of the first three rows by hand in a text editor, using section 2.5 as your model.

Steps:

1. **Open each file in a plain-text editor.** Which ones can you read? For the `.xlsx`, what do you see, and why?
2. **Find the missing values.** How does each format show a blank sales rep or delivery note? Can you tell "unknown" from "empty text" in each?
3. **Find the types.** In which formats can you tell that `net_revenue` is a number and `order_date` is a date, just by looking?
4. **Import the CSV into Excel or Google Sheets twice:** once by double-clicking, once through the import screen with every column set to Text. Add a row with a code like `00731` to your copy first. What differs?
5. **Compare sizes.** Record each file's size in bytes. Which is smallest? Why is the Excel file larger than the CSV for so little data?
6. **Break a CSV on purpose.** Remove the quotes around `"Gate 2, Okhla Phase II"`, save, and reimport. What happens to the columns?
7. **Write it up.** A one-page comparison table: format, readable as text, keeps types, shows blanks as, size, and "I would use it for...". Finish with three sentences of advice for a colleague who is about to email a CSV.

Stretch goals:

- Save the same data as a ZIP file and compare its size with the CSV.
- Type ₹ into one value in your spreadsheet, save it once as CSV UTF-8 and once as plain CSV (*Comma delimited*), then open both files in a text editor. Does the symbol survive in each?
- Check which of your own accounts have MFA switched on, and switch it on for the rest.

You've got it when...

- ☐ I can explain bits, bytes, and why ₹ takes more space than R.
- ☐ I can convert between KB, MB, GB, and TB, and between Mbps and MB/s, and I know why a 1 TB drive shows 931 GB.

- ☐ I know the difference between memory and storage, and what "saved" means for my work.
 - ☐ I name files so they sort by date and never need the word "final".
 - ☐ I can say which format I'd use for a person, an app, or a large dataset, and why.
 - ☐ I import CSV files instead of double-clicking them when codes or IDs are involved.
 - ☐ I can explain a server, the cloud, and an API to a non-technical colleague.
 - ☐ I can read an API response and tell from its status code whether the problem is mine or the server's.
 - ☐ My own accounts use unique passwords from a password manager, with MFA.
 - ☐ I can explain the 3-2-1 rule and why syncing isn't a backup.
-

Recap

- Computers store everything as **bits** (0s and 1s), grouped into **bytes**. Text is stored through an **encoding** such as **UTF-8**; the wrong encoding produces garbled characters. Decimals are usually stored approximately, which is why money needs exact types.
 - Sizes go **KB → MB → GB → TB → PB**, each 1,000 times the last (or 1,024 in some software). Speeds are in **bits**, sizes in **bytes**: divide Mbps by 8.
 - **Memory** is fast and temporary; **storage** is slower and permanent. Unsaved work lives only in memory.
 - Files have **paths** and **extensions**. Name them date-first, never "final", and watch for hidden extensions.
 - **CSV** is universal but has no types; **Excel** is for people; **JSON** and **XML** carry labeled data between systems; **PDF** is for reading; **Parquet** is compact, typed, and fast for large-scale analysis. Import CSVs carefully.
 - **Databases** let many people use data at once, with rules and SQL.
 - A **server** answers a **client's** requests over the **internet**; **HTTPS** encrypts the connection; the **cloud** is rented computers and services (IaaS, PaaS, SaaS).
 - An **API** is the waiter between programs: a request goes in, a response with a **status code** and usually JSON comes back. API keys are passwords.
 - Protect **confidentiality, integrity, and availability**: unique passwords, MFA, least privilege, encryption, hashes, tested **3-2-1 backups**, version history, careful handling of personal data, and reporting mistakes immediately.
-

Practice exercises

Warm-up

1. Convert: (a) 3.5 GB to MB; (b) 250,000 KB to MB; (c) 2 TB to GB.

2. How many bytes does each take in UTF-8: `Riverstone` , `₹500` ?
3. A friend's new laptop has "512 GB" of storage, but Windows shows about 476 GB. Is anything wrong? Explain in two sentences.
4. What file type is each of these, and which program would normally open it: `2026-03-31_stock.csv` , `invoice_9007.pdf` , `po_template.xlsm` , `export.json` , `setup.exe` ?

Core

5. How long, at best, does it take to download a 2 GB file on a 50 Mbps connection?
6. Choose the best format for each job, and say why in one line: (a) a monthly summary your manager will open and filter; (b) a website sending a new order to the sales system; (c) two years of sensor readings from Riverstone's machines, to be analyzed in the cloud; (d) a tax invoice sent to a customer; (e) a one-off list of 3,000 customers to import into a new email tool.
7. This line comes from a CSV file with seven columns. What's wrong with it, and how should it be written? `5010,Green Leaf Hotels,2026-03-03,Delivered,Farah Khan,20100.00,Leave at reception,back gate`
8. A script calls Riverstone's API and gets these replies on different days: `401` , `404` , `429` , `503` . For each, say whether the problem is probably on the script's side or the server's, and what you'd do.
9. Section 2.9 shows two SHA-256 hashes. A colleague sends you a backup file and its hash. You calculate the hash of the file you received and get a different value. What does that tell you, and what doesn't it tell you?

Stretch

10. Kavya keeps her only copy of her spending logs in a Google Drive folder that syncs to her laptop. Does her setup meet the 3-2-1 rule? Describe one thing that could lose her data anyway, and a change that would fix it.
11. Riverstone's new product codes look like `00731` . Describe exactly how a code can lose its leading zeros between the billing system's CSV export and the price lookup in a workbook, and give two ways to prevent it.
12. A 12-megapixel photo is 36 MB before compression, but the file on the phone is 3 MB. Could a sales ledger be compressed the same way? Explain using the terms lossless and lossy.

Think about it (no calculation needed)

13. A team shares one login for the company's CRM, and the password is pinned in a group chat. List three risks, and propose a better setup.
14. Your manager asks, "Should we keep our sales database on our own server in the office, or move it to the cloud?" Give two advantages of each option, and one question you'd ask before deciding.
15. You receive an email from "Riverstone Accounts" with an attachment called `Payment_Details.pdf` and the message "Urgent: confirm today or the supplier contract will be cancelled." What do you check before opening it?

Key terms

bit · byte · binary · ASCII · Unicode · UTF-8 · encoding · garbled text (mojibake) · floating point · pixel · kilobyte (KB) · megabyte (MB) · gigabyte (GB) · terabyte (TB) · petabyte (PB) · KiB / MiB / GiB · megabits per second (Mbps) · memory (RAM) · storage (SSD, hard disk) · file · folder / directory · path · extension · CSV · Excel workbook (.xlsx) · JSON · XML · PDF · Parquet · columnar storage · compression · lossless · lossy · database · database server · client · server · data center · internet · IP address · DNS · HTTP / HTTPS · packet · cloud · IaaS · PaaS · SaaS · region · API · request · response · header · body · status code · API key · rate limit · webhook · CIA triad · confidentiality · integrity · availability · password manager · multi-factor authentication (MFA) · phishing · least privilege · encryption in transit · encryption at rest · hash · SHA-256 · ransomware · backup · 3-2-1 rule · sync · version history · personal data

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Chapter 3, How a Business Runs on Data**, follows one Riverstone order through every system that stores and passes along its data.
 - **Chapters 10 and 11** teach spreadsheets properly, including importing CSV files without damage.
 - **Chapter 12, Databases & SQL Foundations**, turns the one-page preview in section 2.6 into a full, hands-on skill.
 - **Chapter 18** reads CSV, Excel, JSON, and Parquet files in Python, and calls real APIs.
 - **Chapter 20** automates the Friday report from this chapter, including storing API keys safely.
 - **Part V (Chapters 45–52)** builds on formats, compression, the cloud, and APIs at company scale; **Chapter 64** covers security, privacy, and governance in depth.
 - **Interview preparation:** file formats, APIs, and data security questions appear in the Data Engineering bank (Chapter 77) and the Automation & Integration bank (Chapter 78).
-

Answers to practice exercises

(In the finished book these move to Appendix G.)

1. (a) $3.5 \times 1,000 = \mathbf{3,500\ MB}$. (b) $250,000 \div 1,000 = \mathbf{250\ MB}$. (c) $2 \times 1,000 = \mathbf{2,000\ GB}$. (Using binary units instead, the answers would be 3,584 MiB, about 244 MiB, and 2,048 GiB. Either is acceptable if you say which you used.)
2. **Riverstone** is 10 plain English letters: **10 bytes**. **₹500** is **6 bytes**: 3 for ₹ and 1 for each digit.
3. Nothing is wrong. The maker counts 512 GB as 512,000,000,000 bytes, while Windows divides by $1,024^3$ and shows about 476.8 "GB", which is really GiB; it's the same storage measured in a different unit.

4. `2026-03-31_stock.csv` : a CSV table as plain text; a spreadsheet or text editor. `invoice_9007.pdf` : a PDF document; a PDF reader or browser. `po_template.xlsm` : an Excel workbook that can contain macros; Excel (be careful enabling macros from unknown senders). `export.json` : JSON data; a text editor, code editor, or a program that reads JSON. `setup.exe` : a Windows program that runs when opened; only open it if you trust exactly where it came from.

5. $50 \text{ Mbps} \div 8 = 6.25 \text{ MB per second}$. $2,000 \text{ MB} \div 6.25 = \mathbf{320 \text{ seconds, about 5.3 minutes}}$, and in practice a little longer.

6. (a) **Excel**: people will read and filter it, and it keeps formatting and types. (b) **JSON through an API**: it's what apps exchange, and it can carry an order with its lines nested inside. (c) **Parquet**: large, typed, compressed, and fast to analyze by column. (d) **PDF**: a fixed document the customer reads and keeps, which looks the same everywhere. (e) **CSV**: nearly every tool imports it; set the ID and phone columns to text.

7. The delivery note contains a comma but isn't in quotes, so the line splits into **eight** values instead of seven: `Leave at reception` and `back gate` become separate columns, and every program reading it will either reject the row or shift the data. Correct version: `5010,Green Leaf Hotels,2026-03-03,Delivered,Farah Khan,20100.00,"Leave at reception, back gate"`

8. `401 Unauthorized` : the script's side. The API key is missing, wrong, or expired, so check how the key is stored and whether it has been changed. `404 Not Found` : usually the script's side: the ID or the address is wrong, or the record really doesn't exist. Check the ID before assuming the API is broken. `429 Too Many Requests` : the script's side. It's calling too often, so add a pause between calls and retry later. `503` (a 5xx code): the server's side. Retry after a delay, and if it continues, check the provider's status page or contact the API's owner.

9. Different hashes mean **the two files are not identical**: the file was changed or damaged somewhere between your colleague and you, or they calculated the hash on a different version. It doesn't tell you **what** changed, **where**, or **whether** the change was accidental or deliberate. Ask for the file again, compare the hashes, and don't use the backup until they match.

10. No. She has one copy that appears in two places, not three independent copies, and nothing is kept separately from her synced accounts. If she deletes a file, saves over it by mistake, or ransomware encrypts her laptop, the change **syncs to Drive** too. Version history might rescue her, but only for a limited time. A fix: a regular backup to an external drive kept at home, plus a second copy in a separate cloud backup service or account that doesn't sync changes automatically, and an occasional test that she can restore a file.

11. The billing system writes `00731` correctly into the CSV as text. When someone **double-clicks** the CSV, Excel guesses that `00731` is a number and stores **731**. If the file is saved, the damage is written back. The price list still says `00731`, so a lookup for `731` finds nothing. Prevention: (1) import the CSV with *Data → From Text/CSV* and set the product code column to **Text** before loading; (2) better still, get the data from the source system through a connection or an API that keeps the column's type, so no one opens the raw CSV at all. (Adding a letter prefix to codes, such as `P00731`, also prevents it, but changing codes is a business decision.)

12. Only with **lossless** compression. A photo can use **lossy** compression (JPEG) because discarding tiny details people won't notice doesn't matter. A sales ledger can't lose a single digit, so it must use lossless compression, such as ZIP, which restores exactly the original bytes. Ledgers compress very well anyway, because columns such as status, product, and customer repeat the same values many times: the 24.1 MB CSV in section 2.5 zipped to 4.7 MB with nothing lost.

13. Risks: (1) nobody can tell who made a change or deleted a record, so mistakes and misuse can't be traced; (2) when someone leaves the team, they still know the password, and removing their access means changing it for everyone; (3) anyone who can read the chat, on any device, including a lost phone, can log in; and a single leaked password exposes the whole CRM. Better: one account per person, with permissions matched to each role (least privilege), MFA switched on, and access removed promptly when people leave.

14. *Own server:* direct physical control of where the data is; can work on the office network even when the internet is down. *Cloud:* no hardware to buy or maintain, with backups, updates, and failover handled by the provider; easy to scale up as data grows, and reachable securely from anywhere. Useful questions include: "What happens if our office server fails at 2 a.m., and who fixes it?", "Are there rules about which country our customer data must stay in?", and "What would the cloud cost each month at our size, compared with buying and running our own server?"

15. Check the **sender's real email address**, not just the display name; check whether the attachment is really a PDF, with extensions visible, because `Payment_Details.pdf.exe` is a program; notice the **pressure** ("urgent", "today", "cancelled"), a classic phishing sign; and **confirm by phone**, using a number you already have rather than one in the email, before opening the attachment or acting on it. If in doubt, report it to IT without opening it.